

SISTEMAS
DISTRIBUIDOS

Práctica 7.

PWA.

MICHELLE ANGUIANO RODEA

ESCOM

PROFESOR CHADWICK CARRETO
ARELLANO

GRUPO: 7CM6



Instituto
politécnico
nacional

ESCOM

Introducción

Esta práctica extiende el servicio REST desarrollado en la práctica anterior para convertirlo en una Aplicación Web Progresiva (PWA) denominada PWA Books. La aplicación permite gestionar libros (listar y agregar) en un entorno cliente–servidor escrito en Java puro con `com.sun.net.httpserver`. Se incorporan capacidades offline e instalación gracias a dos componentes clave de las PWA: Service Worker (estrategias de caché) y Web App Manifest (metadatos e instalación). El comportamiento offline se implementa guardando una copia de los datos en `LocalStorage` cuando hay red y usando esa copia cuando no hay conexión; la interfaz y estilos se cargan desde la caché del Service Worker.

Problemática

- Disponibilidad sin conexión: la app debe funcionar cuando el usuario pierde Internet (modo lectura).
- Simplicidad tecnológica: sin frameworks web adicionales; únicamente Java + HTML/CSS/JS.
- Persistencia ligera: almacenamiento persistente y legible (JSON plano) en `data/books.json`.
- Instalación tipo app: manifest + iconos (192×192 y 512×512) para permitir la instalación.
- Seguridad básica: evitar path traversal al servir estáticos, CORS para la API y escape en el cliente.
- Mantenibilidad: estructura clara y versionado de caché para actualizaciones.

Solución

Arquitectura cliente–servidor:

Servidor Java (`HttpServer`): sirve '/', archivos de 'static/' y expone API REST:

- GET `/api/books` → lista libros (JSON).
- POST `/api/books` → agrega un libro {titulo, categoria}.
- (Opcional) DELETE `/api/books/{id}` → elimina por id.

Persiste en `data/books.json` y expone `/sw.js` en raíz.

Cliente (PWA):

- Service Worker (sw.js) con cache-first para activos y limpieza de versiones antiguas.
- Manifest (manifest.json) con nombre, colores e iconos 192/512.
- App (index.html, style.css, app.js): en línea consume la API y guarda copia en LocalStorage; en offline renderiza desde LocalStorage y muestra 'sin conexión (solo lectura)'.

Implementación

Estructura del proyecto:

p7-pwa-books/

├─ src/BookServer.java

├─ data/books.json

└─ static/

├─ index.html

├─ style.css

├─ app.js

├─ manifest.json

├─ sw.js

└─ icons/

├─ icon-192.png

└─ icon-512.png

Endpoints:

- GET /api/books → devuelve arreglo JSON de libros (200).
- POST /api/books → crea libro {titulo, categoria} (201, 400 si falta titulo).
- DELETE /api/books/{id} → (opcional) elimina por id (200, 404).

Flujo offline:

- 1) En install, el SW precachea los activos principales.
- 2) En activate, elimina cachés antiguos.
- 3) En fetch, usa cache-first para GET estáticos; la API no se cachea.
- 4) En cliente, online sincroniza LocalStorage; offline renderiza desde LocalStorage.

Compilación y ejecución (Windows):

Con package src; en el archivo:

```
javac -encoding UTF-8 -d out src\BookServer.java
```

```
java -cp ".;out" src.BookServer
```

Sin package:

```
javac -encoding UTF-8 -d out src\BookServer.java
```

```
java -cp ".;out" BookServer
```

Abrir: <http://127.0.0.1:8080/>

```

data > books.json > ...
1 [{"id":1,"categoria":"","titulo":"harry potter"}, {"id":2,"categoria":"","titulo":"it"}] 🐼

1 package src;
2 import com.sun.net.httpserver.*;
3 import java.io.*;
4 import java.net.InetSocketAddress;
5 import java.net.URLDecoder;
6 import java.nio.charset.StandardCharsets;
7 import java.nio.file.*;
8 import java.time.ZonedDateTime;
9 import java.util.*;
10 import java.util.regex.Matcher;
11 import java.util.regex.Pattern;
12
13 public class BookServer {
14     // Config
15     private static final int PORT = 8080;
16     private static final Path STATIC_DIR = Paths.get(first:"static");
17     private static final Path DATA_FILE = Paths.get(first:"data", ...more:"books.json");
18
19     // Estado en memoria (se sincroniza con disco)
20     private static final List<Book> BOOKS = Collections.synchronizedList(new ArrayList<>());
21     private static int NEXT_ID = 1;
22
23     Run | Debug
24     public static void main(String[] args) throws Exception {
25         ensureDataFile();
26         loadFromDisk();
27
28         HttpServer server = HttpServer.create(new InetSocketAddress(PORT), backlog:0);
29
30         // API REST
31         server.createContext(path:"/api/books", exchange -> {
32             addCORS(exchange);
33             try {
34                 switch (exchange.getRequestMethod()) {
35                     case "GET" -> handleListBooks(exchange);
36                     case "POST" -> handleCreateBook(exchange);
37                     case "OPTIONS" -> options(exchange);
38                     default -> sendJSON(exchange, code:405, json:"{\"error\":\"Method not allowed\"}");
39                 }
40             } catch (Exception e) {
41                 e.printStackTrace();
42                 sendJSON(exchange, code:500, json:"{\"error\":\"Internal error\"}");
43             } finally {
44                 exchange.close();
45             }
46         });
47
48         // Service Worker en raíz (content-type correcto)
49         server.createContext(path:"/sw.js", exchange -> {
50             addCORS(exchange);
51             if (!"GET".equals(exchange.getRequestMethod())) {
52                 sendText(exchange, code:405, text:"Method not allowed", contentType:"text/plain");
53                 return;
54             }
55             Path p = STATIC_DIR.resolve(other:"sw.js");
56             sendFile(exchange, p, contentType:"application/javascript");
57         });
58     }
59 }

```

```

});

// Archivos estáticos y SPA fallback
server.createContext(path: "/", exchange -> {
    addCORS(exchange);
    if (!"GET".equals(exchange.getRequestMethod())) {
        sendText(exchange, code:405, text:"Method not allowed", contentType:"text/plain");
        return;
    }
    String rawPath = exchange.getRequestURI().getPath();
    String path = URLDecoder.decode(rawPath, StandardCharsets.UTF_8);
    Path file = STATIC_DIR.resolve(path.substring(beginIndex:1)).normalize();

    if (path.equals(anObject: "/")) file = STATIC_DIR.resolve(other:"index.html");

    // Evitar path traversal
    if (!file.startsWith(STATIC_DIR)) {
        sendText(exchange, code:403, text:"Forbidden", contentType:"text/plain");
        return;
    }

    if (Files.exists(file) && !Files.isDirectory(file)) {
        String ctype = contentType(file);
        sendFile(exchange, file, ctype);
    } else {
        // fallback SPA
        sendFile(exchange, STATIC_DIR.resolve(other:"index.html"), contentType:"text/html; charset=utf-8");
    }
});

server.setExecutor(executor:null);
System.out.println("PWA Books corriendo en http://127.0.0.1:" + PORT);
server.start();
}

// ===== Handlers API =====
private static void handleListBooks(HttpExchange ex) throws IOException {
    String json = booksToJson(BOOKS);
    sendJSON(ex, code:200, json);
}

private static void handleCreateBook(HttpExchange ex) throws IOException {
    String body = new String(ex.getRequestBody().readAllBytes(), StandardCharsets.UTF_8);

    // Parseo JSON muy simple (sin libs): {"categoria":"X" "titulo":"Y"}
    // opcionalmente puede venir "id", se ignora y se reasigna
    Map<String, String> map = parseJsonFlat(body);

    String categoria = opt(map.get(key:"categoria")).trim();
    String titulo = opt(map.get(key:"titulo")).trim();

    if (titulo.isEmpty()) {
        sendJSON(ex, code:400, json:"{\"error\":\"El título es obligatorio\"}");
        return;
    }
}

```

```

        return;
    }
    int id = NEXT_ID++;
    Book b = new Book(id, categoria, titulo);
    BOOKS.add(b);
    saveToDisk();

    sendJSON(ex, code:201, bookToJson(b));
}

private static void options(HttpExchange ex) throws IOException {
    Headers h = ex.getResponseHeaders();
    h.add(key:"Access-Control-Allow-Methods", value:"GET,POST,OPTIONS");
    h.add(key:"Access-Control-Allow-Headers", value:"Content-Type");
    ex.sendResponseHeaders(rCode:204, -1);
}

// ===== Modelo =====
static class Book {
    int id;
    String categoria;
    String titulo;
    Book(int id, String categoria, String titulo) {
        this.id = id; this.categoria = categoria; this.titulo = titulo;
    }
}

// ===== Persistencia simple (JSON plano) =====
private static void ensureDataFile() throws IOException {
    if (!Files.exists(DATA_FILE.getParent())) Files.createDirectories(DATA_FILE.getParent());
    if (!Files.exists(DATA_FILE)) {
        Files.writeString(DATA_FILE, csq:"[]", StandardCharsets.UTF_8);
    }
}

private static synchronized void loadFromDisk() throws IOException {
    String raw = Files.readString(DATA_FILE, StandardCharsets.UTF_8).trim();
    if (raw.isEmpty()) raw = "[]";
    List<Book> tmp = new ArrayList<>();
    // Parseo MUY básico de array de objetos (solo campos id/categoria/titulo)
    Pattern objP = Pattern.compile(regex:"\\{\\{([\\^}]*)\\}\\}\\}");
    Matcher m = objP.matcher(raw);
    int maxId = 0;
    while (m.find()) {
        Map<String,String> map = parseJsonFlat("{" + m.group(group:1) + "}");
        int id = toInt(map.get(key:"id"));
        String categoria = opt(map.get(key:"categoria"));
        String titulo = opt(map.get(key:"titulo"));
        if (id > maxId) maxId = id;
        tmp.add(new Book(id, categoria, titulo));
    }
    BOOKS.clear();
    BOOKS.addAll(tmp);
    NEXT_ID = Math.max(a:1, maxId + 1);
}

```

```

private static synchronized void saveToDisk() throws IOException {
    String json = booksToJson(BOOKS);
    Files.writeString(DATA_FILE, json, StandardCharsets.UTF_8);
}

// ===== Utilidades JSON minimalistas =====
private static Map<String,String> parseJsonFlat(String json) {
    Map<String,String> map = new HashMap<>();
    // extrae "key":"value" o "key":123 (sin anidación)
    Pattern pStr = Pattern.compile(regex:"\"(.*)\"\\s*:\\s*\"(.*)\"");
    Matcher ms = pStr.matcher(json);
    while (ms.find()) {
        map.put(ms.group(group:1), unescape(ms.group(group:2)));
    }
    Pattern pNum = Pattern.compile(regex:"\"(.*)\"\\s*:\\s*(\\d+)");
    Matcher mn = pNum.matcher(json);
    while (mn.find()) {
        map.put(mn.group(group:1), mn.group(group:2));
    }
    return map;
}

private static String escape(String s) {
    return s.replace(target:"\\",replacement:"\\\\").replace(target:"\"",replacement:"\\\"")
        .replace(target:"\n",replacement:"\\n").replace(target:"\r",replacement:"\\r");
}

private static String unescape(String s) {
    return s.replace(target:"\\\\",replacement:"\\").replace(target:"\\\"",replacement:"\"")
        .replace(target:"\\n",replacement:"\n").replace(target:"\\r",replacement:"\r");
}

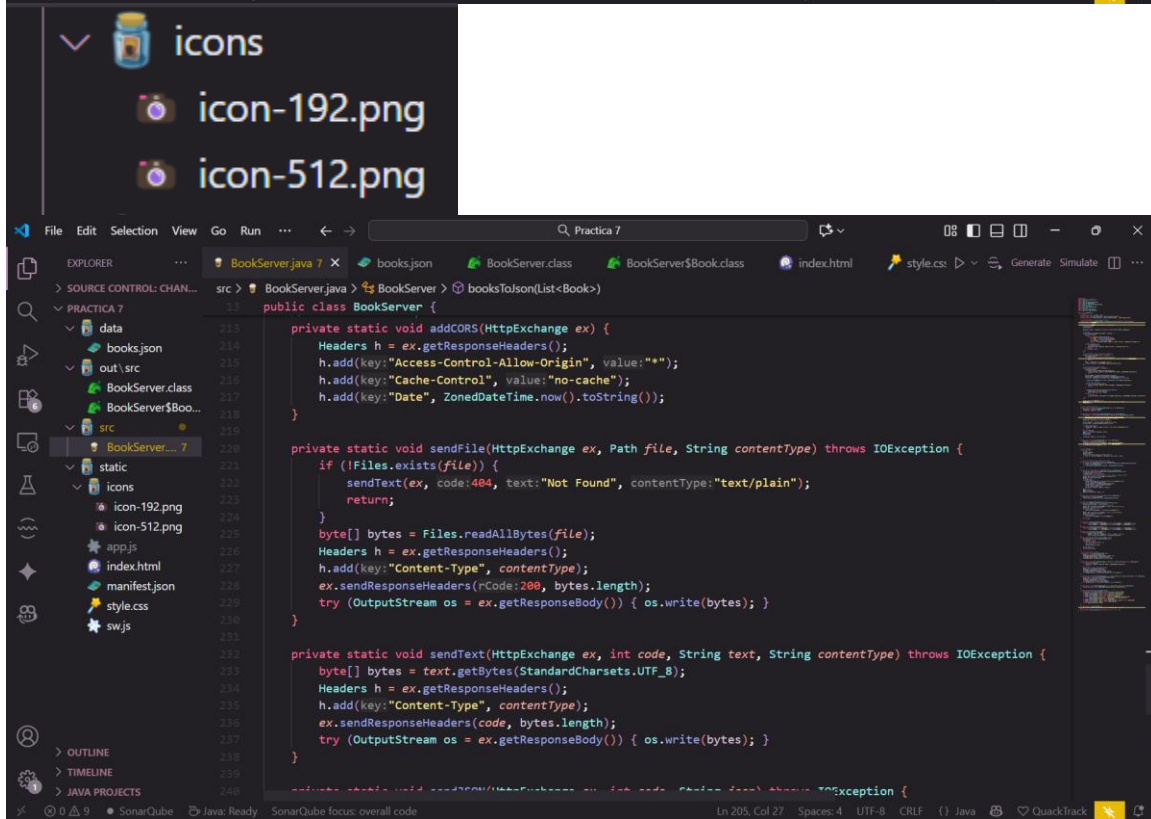
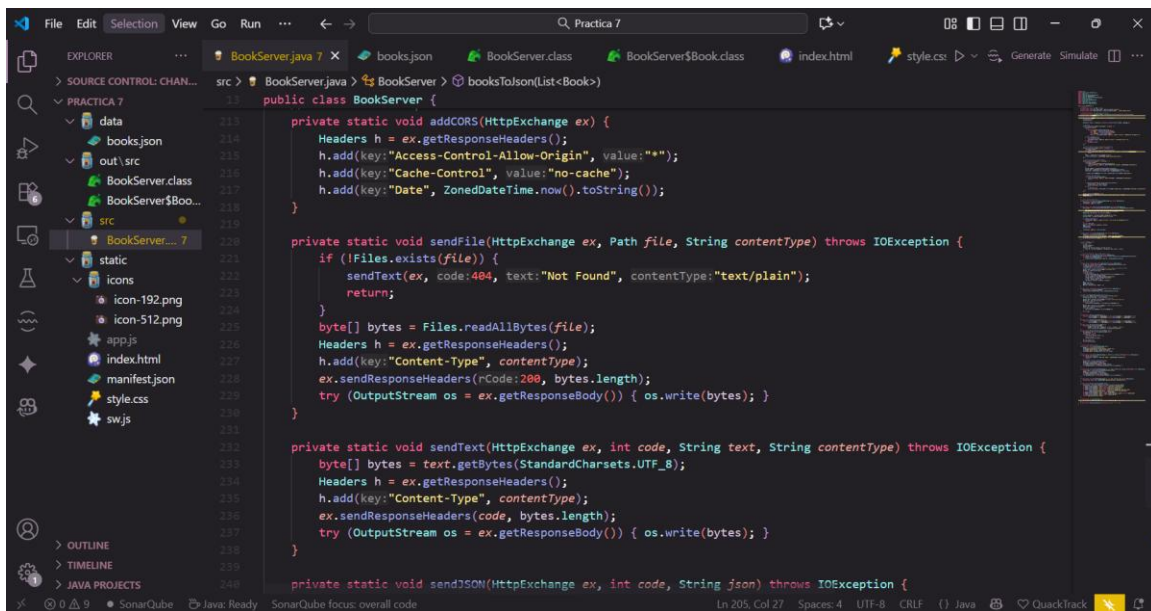
private static String unescape(String s) {
    return s.replace(target:"\\n",replacement:"\n").replace(target:"\\r",replacement:"\r")
        .replace(target:"\\\"",replacement:"\"").replace(target:"\\\\",replacement:"\\");
}

private static String bookToJson(Book b) {
    return String.format(Locale.ROOT,
        format:"{\\\"id\\\":%d,\\\"categoria\\\":\\\"%s\\\",\\\"titulo\\\":\\\"%s\\\"}",
        b.id, escape(opt(b.categoria)), escape(opt(b.titulo)));
}

private static String booksToJson(List<Book> list) {
    StringBuilder sb = new StringBuilder(str:"[");
    boolean first = true;
    for (Book b : list) {
        if (!first) sb.append(str:",");
        first = false;
        sb.append(bookToJson(b));
    }
    sb.append(str:"]");
    return sb.toString();
}

// ===== HTTP helpers =====
private static void addCORS(HttpExchange ex) {
    Headers h = ex.getResponseHeaders();

```

```

private static void addCORS(HttpExchange ex) {
    Headers h = ex.getResponseHeaders();
    h.add(key:"Access-Control-Allow-Origin", value:"*");
    h.add(key:"Cache-Control", value:"no-cache");
    h.add(key:"Date", ZonedDateTime.now().toString());
}

private static void sendFile(HttpExchange ex, Path file, String contentType) throws IOException {
    if (!Files.exists(file)) {
        sendText(ex, code:404, text:"Not Found", contentType:"text/plain");
        return;
    }
    byte[] bytes = Files.readAllBytes(file);
    Headers h = ex.getResponseHeaders();
    h.add(key:"Content-Type", contentType);
    ex.sendResponseHeaders(rCode:200, bytes.length);
    try (OutputStream os = ex.getResponseBody()) { os.write(bytes); }
}

private static void sendText(HttpExchange ex, int code, String text, String contentType) throws IOException {
    byte[] bytes = text.getBytes(StandardCharsets.UTF_8);
    Headers h = ex.getResponseHeaders();
    h.add(key:"Content-Type", contentType);
    ex.sendResponseHeaders(code, bytes.length);
    try (OutputStream os = ex.getResponseBody()) { os.write(bytes); }
}

private static void sendJSON(HttpExchange ex, int code, String json) throws IOException {
    ex.sendResponseHeaders(code, bytes.length);
    try (OutputStream os = ex.getResponseBody()) { os.write(bytes); }
}

private static void sendJSON(HttpExchange ex, int code, String json) throws IOException {
    sendText(ex, code, json, contentType:"application/json; charset=utf-8");
}

private static String contentType(Path file) {
    String name = file.getFileName().toString().toLowerCase(Locale.ROOT);
    if (name.endsWith(suffix:".html")) return "text/html; charset=utf-8";
    if (name.endsWith(suffix:".css")) return "text/css; charset=utf-8";
    if (name.endsWith(suffix:".js")) return "application/javascript";
    if (name.endsWith(suffix:".json")) return name.equals(anObject:"manifest.json")
        ? "application/manifest+json" : "application/json; charset=utf-8";
    if (name.endsWith(suffix:".png")) return "image/png";
    if (name.endsWith(suffix:".jpg") || name.endsWith(suffix:".jpeg")) return "image/jpeg";
    if (name.endsWith(suffix:".svg")) return "image/svg+xml";
    return "application/octet-stream";
}

private static int toInt(String s) {
    try { return Integer.parseInt(s); } catch (Exception e) { return 0; }
}

private static String opt(String s) { return s == null ? "" : s; }

```

```
const API = '/api/books';
const $lista = document.getElementById('lista');
const $form = document.getElementById('form');
const $titulo = document.getElementById('titulo');
const $categoria = document.getElementById('categoria');
const $net = document.getElementById('net'); | 🦆
```

```
let offline = false;
```

```
async function cargar() {
  try {
    const res = await fetch(API, { cache: 'no-store' });
    if (!res.ok) throw new Error('bad');
    const data = await res.json();
    localStorage.setItem('books', JSON.stringify(data));
    pintar(data);
    offline = false;
    $net.textContent = '● conectado';
  } catch (e) {
    const cache = JSON.parse(localStorage.getItem('books') || '[]');
    pintar(cache);
    offline = true;
    $net.textContent = '● sin conexión (solo lectura)';
  }
}
```

```
function pintar(data) {
  $lista.innerHTML = '';
  data.forEach(b => {
```

```

data.forEach(b => {
  const li = document.createElement('li');
  li.innerHTML = `
    <div>
      <strong>#${b.id}</strong> ${escapeHTML(b.titulo)}
      <small style="color:#9aa3c7;margin-left:8px">${escapeHTML(b.categoria || "")}</small>
    </div>
    <button disabled title="Solo lectura">📖</button>
  `;
  $lista.appendChild(li);
});
}

$form.addEventListener('submit', async (e) => {
  e.preventDefault();
  if (offline) { alert('Estás offline: no se puede agregar.');
  return; }
  const titulo = $titulo.value.trim();
  const categoria = $categoria.value.trim();
  if (!titulo) return;
  try {
    const res = await fetch(API, {
      method: 'POST',
      headers: {'Content-Type': 'application/json'},
      body: JSON.stringify({ titulo, categoria })
    });
    if (!res.ok) throw 0;
    $titulo.value = '';
    $categoria.value = '';
  } catch {
    alert('No se pudo agregar el libro.');
  }
});

function escapeHTML(s) {
  return String(s).replace(/[\&<>"']/g, c=>({ '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;' }[c]));
}

cargar();
window.addEventListener('online', cargar);

```

```

<!doctype html>
<html lang="es">
<head>
  <meta charset="utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>P7 • PWA Books (Java HttpServer)</title>
  <link rel="manifest" href="/manifest.json" />
  <meta name="theme-color" content="#0ea5e9" />
  <link rel="stylesheet" href="/style.css" />
</head>
<body>
  <header class="wrap">
    <h1>📖 PWA Books</h1>
    <div id="net" class="net">🕒 comprobando red...</div>
  </header>

  <main class="wrap">
    <section class="card">
      <form id="form" autocomplete="off">
        <input id="titulo" type="text" placeholder="Título del libro..." required />
        <input id="categoria" type="text" placeholder="Categoría (opcional)" />
        <button type="submit">Agregar</button>
      </form>
    </section>

    <section class="card">
      <h2>Listado</h2>
      <ul id="lista" class="lista"></ul>
    </section>
  </main>

  <footer class="wrap foot">
    <small>Instalable + offline: Service Worker (activos estáticos) y LocalStorage (copia de datos).</small>
  </footer>

  <script src="/app.js"></script>
  <script>
    if ('serviceWorker' in navigator) {
      navigator.serviceWorker.register('/sw.js');
    }
  </script>
</body>
</html>

```

```

{
  "name": "P7 • PWA Books",
  "short_name": "Books",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#0b1020",
  "theme_color": "#0ea5e9",
  "icons": [
    { "src": "/icons/icon-192.png", "sizes": "192x192", "type": "image/png" },
    { "src": "/icons/icon-512.png", "sizes": "512x512", "type": "image/png" }
  ]
}

const CACHE = 'p7-books-v1';
const PRECACHE = [
  '/', '/index.html', '/style.css', '/app.js',
  '/manifest.json', '/icons/icon-192.png', '/icons/icon-512.png'
];

// install: precache
self.addEventListener('install', (e) => {
  e.waitUntil(caches.open(CACHE).then(c => c.addAll(PRECACHE)));
  self.skipWaiting();
});

// activate: limpiar caches viejos
self.addEventListener('activate', (e) => {
  e.waitUntil(
    caches.keys().then(keys => Promise.all(keys.filter(k => k !== CACHE).map(k => caches.delete(k))))
  );
  self.clients.claim();
});

// fetch: cache-first para estáticos; dejar pasar API
self.addEventListener('fetch', (e) => {
  const req = e.request;
  const url = new URL(req.url);

  // No interceptar llamadas a /api ni métodos distintos de GET
  if (url.pathname.startsWith('/api/') || req.method !== 'GET') return;

  e.respondWith(

```

```

:root { --bg: #0b1020; --card: #111634; --text: #e8ecff; --muted: #9aa3c7; --accent: #0ea5e9; }
* { box-sizing: border-box; }
html, body { margin: 0; background: var(--bg); color: var(--text); font-family: system-ui, Segoe UI, Roboto, sans-serif; }
.wrap { max-width: 900px; margin: 16px auto; padding: 0 16px; }
h1 { margin: 0 0 6px; }
.net { color: var(--muted); }
.card { background: var(--card); border: 1px solid #1f2758; border-radius: 16px; padding: 16px; margin-bottom: 16px; box
form { display: flex; gap: 8px; }
input { flex: 1; padding: 12px 14px; border-radius: 12px; border: 1px solid #23307b; background: #0e1440; color: var(--
button { padding: 12px 14px; background: var(--accent); color: #06101f; border: none; border-radius: 12px; font-weight: 6
.lista { list-style: none; margin: 0; padding: 0; display: flex; flex-direction: column; gap: 8px; }
li { display: flex; align-items: center; justify-content: space-between; gap: 12px; background: #0f1542; border: 1px soli
.foot { text-align: center; color: var(--muted); }

```

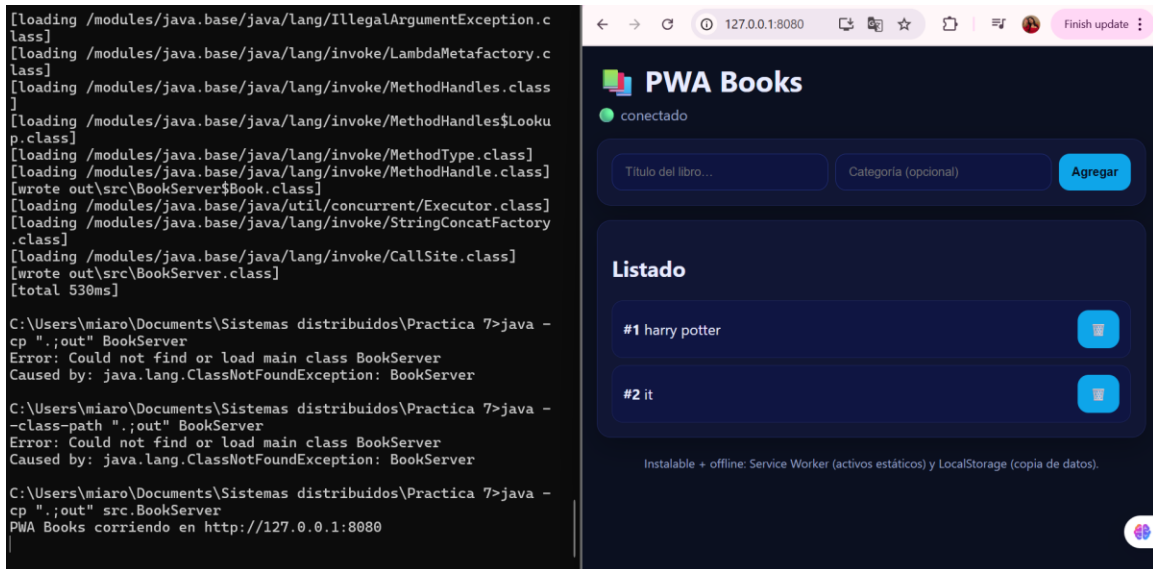
```

e.respondWith(
  caches.match(req).then(cached => {
    if (cached) return cached;
    return fetch(req).then(resp => {
      const clone = resp.clone();
      caches.open(CACHE).then(c => c.put(req, clone));
      return resp;
    }).catch(() => {
      if (req.mode === 'navigate') return caches.match('/index.html');
    });
  });
);
});

```

Resultados

- La app funcionó en <http://127.0.0.1:8080/>, permitiendo agregar libros y listar.
- El archivo `data/books.json` se actualiza tras cada inserción; la API responde JSON.
- El Service Worker quedó 'activated and running'; se observó el caché con activos precargados.
- En modo offline, la UI se sirvió desde caché y el listado mostró la copia de `LocalStorage` con el aviso correspondiente.
- El manifest válido permitió la instalación de la PWA y ejecución en modo standalone.



Conclusión

Se evolucionó el servicio REST a una PWA completa sin frameworks adicionales. La estrategia cache-first para activos y LocalStorage para datos asegura disponibilidad offline (solo lectura) y mejora la experiencia de usuario. Como trabajo futuro se propone añadir sincronización diferida (Background Sync), versionado de caché con notificación de nueva versión y pruebas automatizadas del SW.

Referencias

- MDN Web Docs: Service Workers y Progressive Web Apps.
- W3C: Web App Manifest.
- Oracle JDK: módulo jdk.httpserver (HttpServer, HttpHandler, HttpExchange).
- Apuntes de la Práctica 6 (arquitectura REST y JSON).